

Sistema de Gestión con Base de Datos Relacional y Documental

Santiago Chavarro Herrera
Grupo 10
schavarroo@unbosque.edu.com

Juan Pablo Sánchez
Grupo 10
jpasanchezo@unbosque.edu.com

Resumen—Este artículo presenta el diseño e implementación de un prototipo de sistema de gestión de inventario que integra dos paradigmas de bases de datos: relacional (PostgreSQL) y documental (MongoDB). La arquitectura propuesta aprovecha las fortalezas de cada tecnología — integridad transaccional en el modelo relacional y flexibilidad de esquema en el modelo documental — para construir un sistema robusto, escalable y mantenible. Se describen las decisiones de diseño, el esquema de datos, los mecanismos de automatización mediante triggers y las propuestas de mejora para una versión de producción.

Index Terms—PostgreSQL, MongoDB, base de datos relacional, base de datos documental, triggers, inventario, arquitectura híbrida.

I. INTRODUCCIÓN Y PROPUESTA DEL PROYECTO

En el contexto actual de digitalización de procesos empresariales, las pequeñas y medianas organizaciones requieren sistemas de gestión que les permitan registrar, consultar y analizar su información operativa de manera confiable y eficiente. Sin embargo, la naturaleza heterogénea de los datos — algunos altamente estructurados y transaccionales, otros variables y semiestructurados — plantea un desafío de diseño que no puede resolverse adecuadamente con una única tecnología de base de datos [1].

I-A. Planteamiento del problema

Las soluciones basadas exclusivamente en bases de datos relacionales imponen esquemas rígidos que dificultan el almacenamiento de información variable, como los atributos específicos de lotes de producción o el registro de eventos con metadatos heterogéneos. Por otro lado, las soluciones puramente documentales sacrifican la integridad referencial y las garantías transaccionales necesarias para el control de inventario y la gestión de ventas.

I-B. Propuesta

Este proyecto propone el diseño e implementación de un **sistema de gestión de inventario de alcance prototípico** que demuestre una arquitectura híbrida fundamentada en el principio de persistencia polígota [1]: utilizar la tecnología de base de datos más adecuada para cada tipo de dato dentro del mismo sistema.

El sistema se basa en dos pilares tecnológicos complementarios:

- **PostgreSQL** como base de datos relacional para gestionar la información estructurada y transaccional del negocio: usuarios, productos, clientes, ventas e inventario. Se aprovechan sus capacidades de integridad referencial, restricciones, y especialmente los *triggers*, que permiten automatizar lógica de negocio crítica directamente en la base de datos [4].
- **MongoDB** como base de datos documental para almacenar información flexible y variable: lotes de producción con atributos dinámicos e historial de eventos del sistema. Su modelo de documentos JSON permite registrar datos sin imponer un esquema fijo, adaptándose a la naturaleza cambiante de esta información [3].

I-C. Alcance del prototipo

El prototipo cubre las siguientes funcionalidades:

- Registro y administración de usuarios, productos, clientes y ventas.
- Control automático de inventario mediante triggers en PostgreSQL.
- Registro flexible de lotes de producción con datos variables en MongoDB.
- Historial trazable de eventos del sistema para auditoría.
- Generación de reportes básicos: estado del inventario, ventas por período y productos más vendidos.

La integración entre ambas bases de datos se realiza mediante referencias de identificadores — sin duplicar información — y la lógica de coordinación reside en la capa de aplicación. Este enfoque sienta las bases para una solución de producción escalable, cuyas líneas de mejora se discuten en la sección VII.

I-D. Justificación tecnológica

La selección de PostgreSQL y MongoDB responde a criterios técnicos concretos: PostgreSQL es uno de los motores relacionales de código abierto más robustos y con mayor soporte de la comunidad [2], mientras que MongoDB es el sistema de base de datos documental más ampliamente adoptado en la industria, con un modelo de datos que se ajusta naturalmente a los requerimientos de flexibilidad del sistema propuesto [3].

II. DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema se compone de dos capas de persistencia integradas:

- **Capa relacional (PostgreSQL):** gestiona la información estructurada y transaccional — usuarios, productos, clientes, ventas e inventario — garantizando integridad referencial mediante claves primarias, foráneas y restricciones.
- **Capa documental (MongoDB):** almacena datos flexibles que no encajan naturalmente en tablas — lotes de producción con atributos variables e historial de eventos del sistema.

La integración entre ambas capas se realiza mediante referencias de identificadores: los documentos en MongoDB incluyen campos `producto_id` o `referencia_id` que apuntan a registros en PostgreSQL, evitando duplicación de información.

III. BASE DE DATOS RELACIONAL — POSTGRESQL

III-A. Esquema de tablas

El modelo relacional está compuesto por seis tablas cuyas responsabilidades se describen en la Tabla I.

Cuadro I
TABLAS DEL MODELO RELACIONAL

Tabla	Descripción
usuario	Gestiona el acceso al sistema. Cada usuario administra productos.
cliente	Personas que realizan compras. Se relacionan con las ventas.
producto	Artículos disponibles para venta. Pertenece a un usuario (FK).
inventario	Control de stock por producto. Relación 1:1 con producto.
venta	Registra transacciones. El total se calcula automáticamente.
detalle_venta	Tabla de unión que resuelve la relación N:M entre ventas y productos.

III-B. Relaciones principales

- Un *usuario* gestiona uno o varios *productos* (1:N).
- Un *producto* tiene exactamente un registro de *inventario* (1:1).
- Un *cliente* puede realizar múltiples *ventas* (1:N).
- Una *venta* contiene uno o varios *productos* a través de *detalle_venta* (N:M).

III-C. Definición del esquema

A continuación se presenta la definición de las tablas principales:

Listing 1. Tablas principales del modelo relacional

```
1 CREATE TABLE usuario (  
2   id SERIAL PRIMARY KEY,  
3   nombre VARCHAR(100) NOT NULL,  
4   email VARCHAR(150) NOT NULL UNIQUE,  
5   password VARCHAR(255) NOT NULL,
```

```
6   created_at TIMESTAMP DEFAULT NOW()  
7 );  
8  
9 CREATE TABLE producto (  
10  id SERIAL PRIMARY KEY,  
11  usuario_id INT NOT NULL  
12  REFERENCES usuario(id) ON DELETE RESTRICT,  
13  nombre VARCHAR(150) NOT NULL,  
14  descripcion TEXT,  
15  precio NUMERIC(10,2) NOT NULL CHECK (precio >= 0)  
16 );  
17  
18 CREATE TABLE inventario (  
19  id SERIAL PRIMARY KEY,  
20  producto_id INT NOT NULL UNIQUE  
21  REFERENCES producto(id) ON DELETE  
22  CASCADE,  
23  cantidad INT NOT NULL DEFAULT 0,  
24  cantidad_minima INT NOT NULL DEFAULT 5,  
25  updated_at TIMESTAMP DEFAULT NOW()  
26 );  
27  
28 CREATE TABLE venta (  
29  id SERIAL PRIMARY KEY,  
30  cliente_id INT NOT NULL  
31  REFERENCES cliente(id) ON DELETE RESTRICT,  
32  fecha TIMESTAMP DEFAULT NOW(),  
33  total NUMERIC(12,2) DEFAULT 0,  
34  estado VARCHAR(20) DEFAULT 'pendiente'  
35  CHECK (estado IN  
36  ('pendiente', 'completada', 'cancelada'))  
37 );  
38  
39 CREATE TABLE detalle_venta (  
40  id SERIAL PRIMARY KEY,  
41  venta_id INT NOT NULL  
42  REFERENCES venta(id) ON DELETE CASCADE,  
43  producto_id INT NOT NULL  
44  REFERENCES producto(id) ON DELETE  
45  RESTRICT,  
46  cantidad INT NOT NULL CHECK (cantidad  
47  > 0),  
48  precio_unitario NUMERIC(10,2) NOT NULL  
49 );
```

IV. TRIGGERS IMPLEMENTADOS

Los triggers son funciones que PostgreSQL ejecuta automáticamente ante eventos de modificación en las tablas. Se implementaron cuatro triggers que encapsulan la lógica de negocio directamente en la base de datos, garantizando su ejecución independientemente del cliente que acceda al sistema [2].

IV-A. *trg_inventario_updated*

Evento: BEFORE UPDATE sobre inventario.

Propósito: actualiza automáticamente el campo `updated_at` con la fecha y hora exacta de cada modificación al stock, sin requerir intervención del backend.

Listing 2. Trigger de actualización de timestamp

```
1 CREATE OR REPLACE FUNCTION fn_inventario_updated()  
2 RETURNS TRIGGER AS $$  
3 BEGIN  
4   NEW.updated_at = NOW();  
5   RETURN NEW;  
6 END;  
7 $$ LANGUAGE plpgsql;
```

IV-B. *trg_descontar_stock*

Evento: AFTER INSERT sobre detalle_venta.

Propósito: al registrar un producto en una venta, descuenta automáticamente la cantidad vendida del inventario. Si el stock

disponible es insuficiente, cancela la transacción completa lanzando una excepción, protegiendo la integridad del inventario.

Listing 3. Trigger de descuento de stock

```
1 CREATE OR REPLACE FUNCTION fn_descontar_stock()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     stock_actual INT;
5 BEGIN
6     SELECT cantidad INTO stock_actual
7     FROM inventario
8     WHERE producto_id = NEW.producto_id;
9
10    IF stock_actual < NEW.cantidad THEN
11        RAISE EXCEPTION
12        'Stock_insuficiente._Producto_id=%,
13        stock_actual=%,_pedido=%',
14        NEW.producto_id, stock_actual, NEW.cantidad;
15    END IF;
16
17    UPDATE inventario
18    SET cantidad = cantidad - NEW.cantidad
19    WHERE producto_id = NEW.producto_id;
20
21    RETURN NEW;
22 END;
23 $$ LANGUAGE plpgsql;
```

IV-C. *trg_recalcular_total*

Evento: AFTER INSERT OR UPDATE OR DELETE sobre detalle_venta.

Propósito: recalcula el campo total de la venta sumando cantidad $\times$ precio_unitario de todos sus detalles cada vez que se agrega, modifica o elimina un ítem.

Listing 4. Trigger de recálculo de total de venta

```
1 CREATE OR REPLACE FUNCTION fn_recalcular_total()
2 RETURNS TRIGGER AS $$
3 DECLARE v_id INT;
4 BEGIN
5     v_id := COALESCE(NEW.venta_id, OLD.venta_id);
6     UPDATE venta
7     SET total = (
8         SELECT COALESCE(SUM(cantidad * precio_unitario), 0)
9         FROM detalle_venta WHERE venta_id = v_id
10    )
11    WHERE id = v_id;
12    RETURN COALESCE(NEW, OLD);
13 END;
14 $$ LANGUAGE plpgsql;
```

IV-D. *trg_alerta_stock*

Evento: AFTER UPDATE sobre inventario.

Propósito: tras cada actualización de stock, verifica si la cantidad actual bajó del mínimo permitido y emite un aviso (NOTICE) que el backend puede capturar para disparar notificaciones.

Listing 5. Trigger de alerta de stock mínimo

```
1 CREATE OR REPLACE FUNCTION fn_alerta_stock()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.cantidad <= NEW.cantidad_minima THEN
5         RAISE NOTICE
6         'ALERTA_STOCK_BAJO._Producto_id=%,
7         actual=%,_minimo=%',
8         NEW.producto_id,
9         NEW.cantidad,
10        NEW.cantidad_minima;
11    END IF;
12    RETURN NEW;
13 END;
14 $$ LANGUAGE plpgsql;
```

La Tabla II resume los cuatro triggers implementados.

Cuadro II
RESUMEN DE TRIGGERS IMPLEMENTADOS

Trigger	Evento	Propósito
trg_inventario_updated	BEFORE UPDATE	Actualiza timestamp de modificación
trg_descontar_stock	AFTER INSERT	Descuenta stock y valida disponibilidad
trg_recalcular_total	AFTER I/U/D	Recalcula total de la venta
trg_alerta_stock	AFTER UPDATE	Emite aviso de stock bajo el mínimo

V. BASE DE DATOS DOCUMENTAL — MONGODB

MongoDB gestiona los datos que, por su naturaleza variable, no se adaptan a un esquema relacional rígido [3]. Se definieron dos colecciones.

V-A. *lotas_produccion*

Almacena cada lote fabricado o recibido de un producto. El campo `datos_variables` es un objeto libre que permite registrar atributos específicos según el tipo de producto (temperatura, certificaciones, número de factura, etc.) sin alterar la estructura general del documento.

Campos principales: `producto_id`, `fecha`, `cantidad_producida`, `calidad`, `origen`, `notas`, `datos_variables`.

V-B. *eventos_historial*

Funciona como un log inmutable de auditoría del sistema. Registra qué ocurrió, cuándo, qué usuario lo hizo y a qué registro de PostgreSQL hace referencia.

Tipos de evento: `produccion`, `venta`, `ajuste_inventario`, `devolucion`, `alerta_stock`.

Campos principales: `tipo`, `fecha`, `descripcion`, `referencia_id`, `producto_id`, `usuario_id`, `metadata`.

VI. REPORTES

El sistema expone reportes a través de vistas en PostgreSQL y consultas de agregación en MongoDB, evitando la necesidad de lógica compleja en el backend:

- **v_inventario:** estado del stock con alerta automática cuando baja del mínimo.
- **v_ventas_detalle:** ventas con nombre de cliente, productos y totales.
- **v_top_productos:** productos más vendidos en unidades e ingresos (solo ventas completadas).
- **Historial de producción:** consulta sobre `lotas_produccion` filtrada por `producto_id`.
- **Log de eventos:** historial completo desde `eventos_historial`, filtrable por tipo o rango de fechas.

VII. PROPUESTAS DE MEJORAMIENTO

Una vez validado el prototipo, se identifican las siguientes líneas de mejora para una versión de producción:

VII-A. Seguridad y autenticación

Implementar autenticación basada en JWT (*JSON Web Tokens*), hashéo de contraseñas con *bcrypt* y un sistema de roles (administrador, vendedor, supervisor) con permisos diferenciados.

VII-B. Backend y API REST

Desarrollar una API REST con Node.js/Express o Python-/FastAPI que conecte ambas bases de datos, con validación de datos de entrada, manejo centralizado de errores y paginación en los endpoints de listado.

VII-C. Optimización de base de datos

Implementar particionamiento de la tabla *venta* por fecha para reportes históricos de alto volumen, y un esquema de *soft delete* (campo *deleted_at*) para preservar el historial de registros eliminados.

VII-D. Interfaz y experiencia de usuario

Desarrollar un panel web con gráficas de ventas e inventario en tiempo real, notificaciones automáticas por correo ante alertas de stock y exportación de reportes a PDF y Excel.

VII-E. Calidad y despliegue

Escribir pruebas de integración para los flujos críticos, contenedorizar la aplicación con Docker Compose e implementar un pipeline de CI/CD para automatizar pruebas y despliegues.

VIII. CONCLUSIONES

La arquitectura híbrida propuesta demuestra que la selección adecuada de tecnología de persistencia, basada en las características de cada tipo de dato, produce sistemas más robustos y mantenibles que una solución monolítica. Los triggers implementados en PostgreSQL encapsulan lógica de negocio crítica directamente en la base de datos, garantizando consistencia independientemente del cliente. MongoDB complementa el sistema al absorber la variabilidad de los datos de producción e historial sin imponer un esquema rígido. El prototipo sienta las bases para una solución de producción escalable con las mejoras identificadas.

REFERENCIAS

- [1] M. Fowler y P. Sadalage, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012.
- [2] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2023. [En línea]. Disponible en: <https://www.postgresql.org/docs/16/>
- [3] K. Chodorow, *MongoDB: The Definitive Guide*, 2.^a ed. O'Reilly Media, 2013.
- [4] C. J. Date, *An Introduction to Database Systems*, 8.^a ed. Addison-Wesley, 2004.